REDES NEURONAIS ARTIFICIAIS

Luís Morgado

ISEL-DEI

REDES NEURONAIS MULTICAMADA

- Uma rede neuronal multicamada é uma rede neuronal artificial composta por múltiplas camadas de neurónios interligadas, que tem a capacidade de discriminar dados não separáveis linearmente
- Um exemplo deste tipo de redes é o *Perceptrão Multicamada*, o qual é uma evolução do modelo de *Perceptrão* base de modo a ultrapassar a limitação de discriminação apenas de dados linearmente separáveis
- Uma rede neuronal artificial multicamada é composta por três tipos de camadas:
  - **Camada de entrada**
    - Regista e disponibiliza os dados de entrada
  - **Camada escondida** (podem ser várias)
    - Processa os dados recebidos da camada anterior
  - **Camada de saída**
    - Produz as saídas da rede com base no processamento das camadas anteriores

REDES NEURONAIS MULTICAMADA

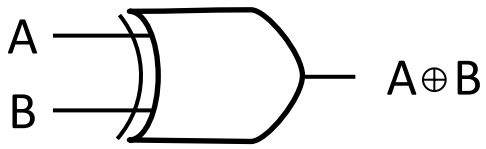
Exemplo: Implementação da disjunção exclusiva (XOR)

Codificação binária

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

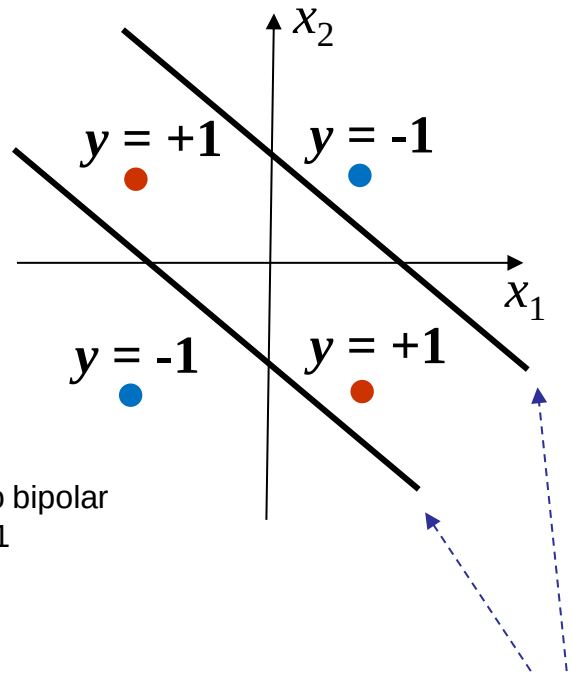
Codificação bipolar

A	B	A⊕B
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1

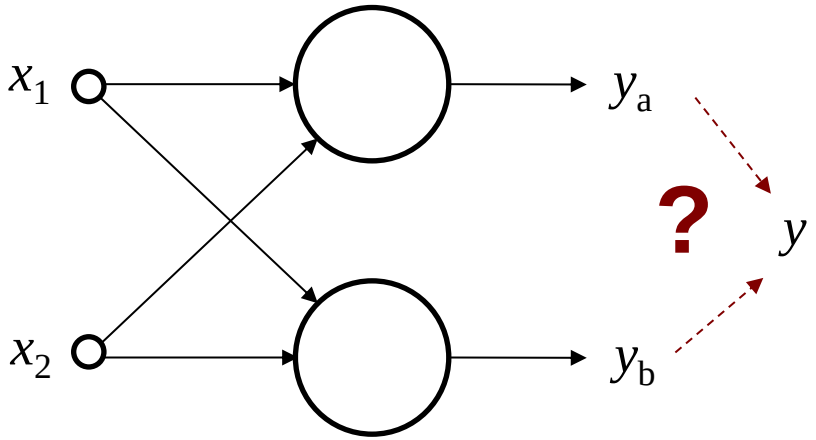


REDES NEURONAIS MULTICAMADA

Disjunção exclusiva (*XOR*)



Codificação bipolar
Verdade: +1
Falso: -1



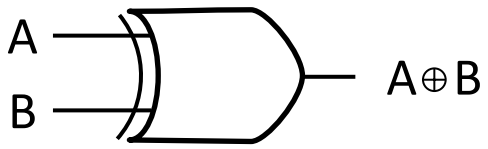
A disjunção exclusiva (XOR) não é possível de realizar com uma única camada de neurónios, mesmo que essa camada seja composta por vários neurónios, pois são necessárias duas fronteiras de separação linear em conjunto para produzir a saída y

REDES NEURONAIS MULTICAMADA

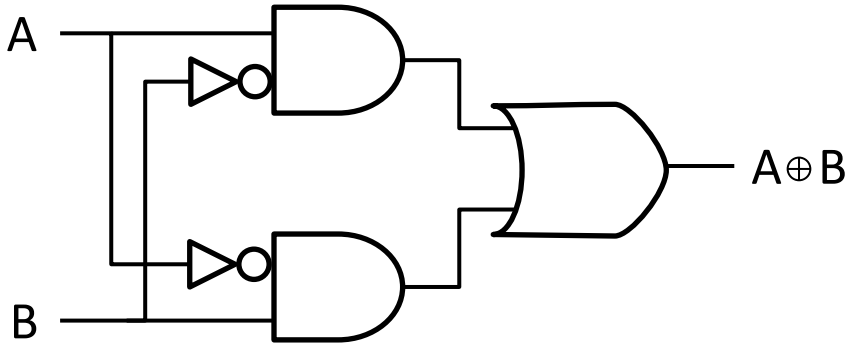
Exemplo: Implementação da disjunção exclusiva (XOR)

Codificação binária

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0



$$A \oplus B = A \cdot \overline{B} + \overline{A} \cdot B$$



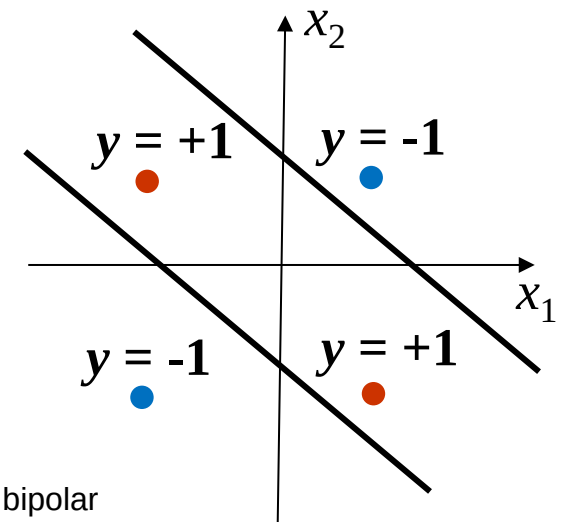
A realização da função XOR requer duas camadas de operadores lógicos

- Uma camada de conjunções (AND)
- Uma camada de disjunções (OR)

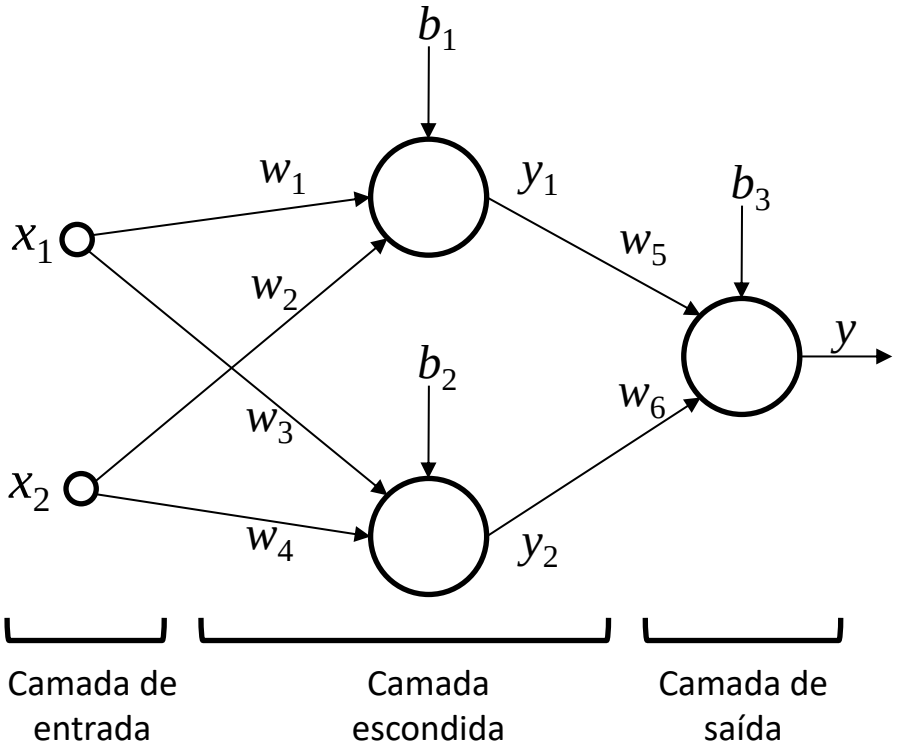
(*Forma Normal Disjuntiva*)

REDES NEURONAIS MULTICAMADA

Disjunção exclusiva (*XOR*)



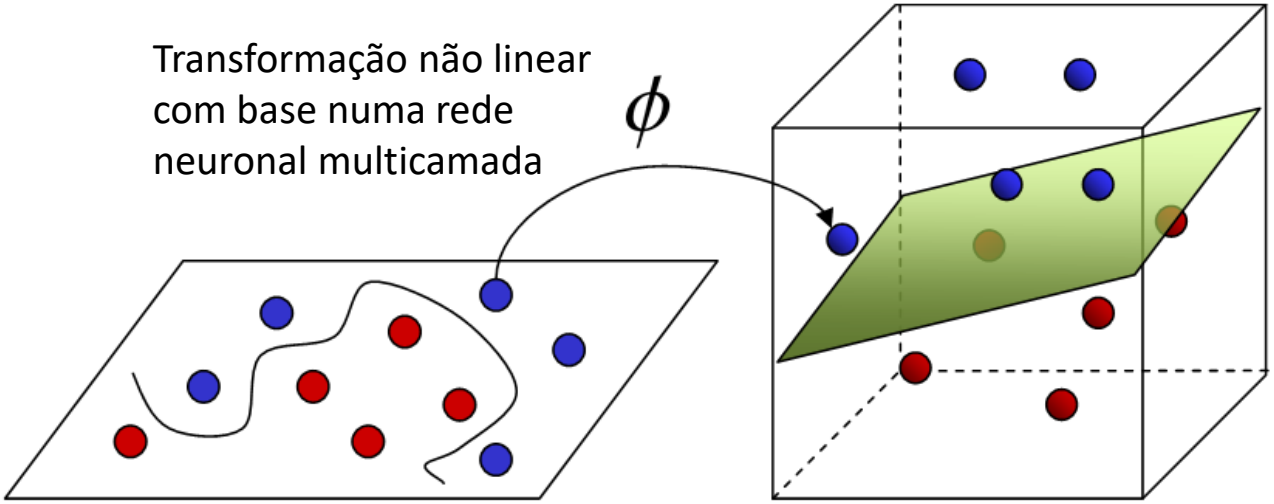
Codificação bipolar
Verdade: +1
Falso: -1



A disjunção exclusiva (XOR) é possível de realizar com uma **camada adicional de neurónios**, que realiza a composição das duas fronteiras de separação linear necessárias para gerar a saída y

REDES NEURONAIS MULTICAMADA

Separabilidade de regiões por transformação do espaço de entrada



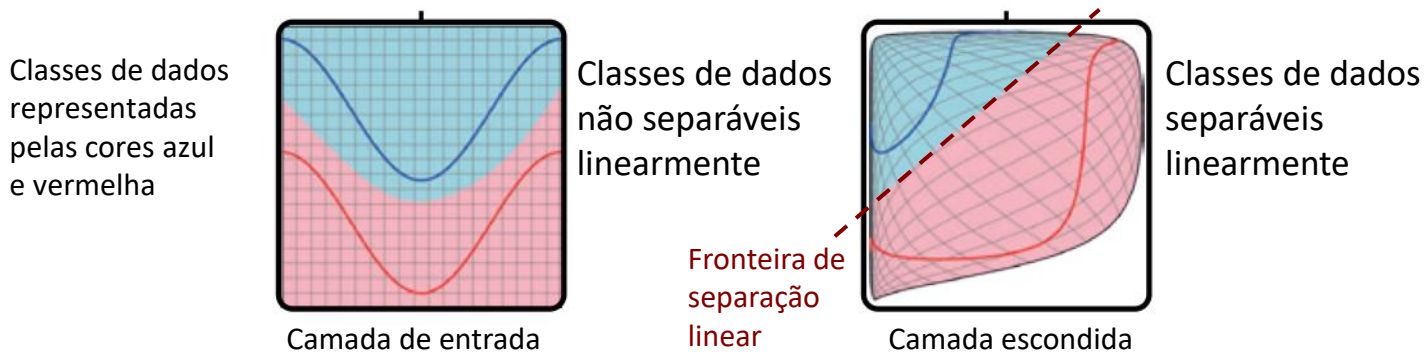
A separação não linear pode ser realizada por transformação do espaço de entrada em espaços de dimensões superiores, onde é possível a separação linear de regiões

As **redes neuronais multicamada** realizam essa transformação através da organização dos neurónios da rede em camadas sucessivas interligadas

REDES NEURONAIS MULTICAMADA

- As camadas escondidas são determinantes para a capacidade de separação não linear das redes neuronais multicamada
- **Cada camada escondida aplica uma transformação não linear aos dados de entrada**, projetando-os num espaço de maior dimensionalidade
- Essa transformação não linear é realizada pelas funções de activação aplicadas em cada neurónio, por exemplo, a função sigmoide ou a função tangente hiperbólica, entre outras
- **Nas camadas escondidas da rede são criadas representações de forma cada vez mais complexa e não linear**, definindo fronteiras de discriminação não lineares no espaço de dados de entrada

Exemplo:



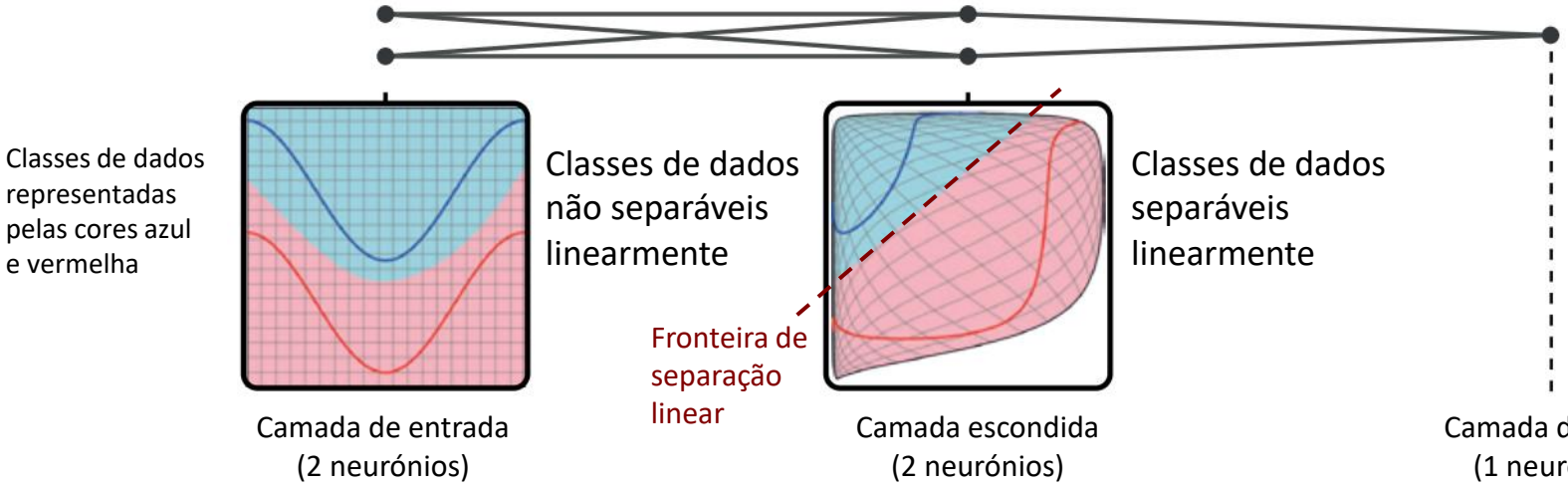
Adaptado de [LeCun *et al.*, 2015]

REDES NEURONAIS MULTICAMADA

Exemplo:

A camada escondida da rede neuronal projeta os dados num espaço de maior dimensionalidade, onde esses dados se tornam separáveis linearmente

A camada de saída combina essa representação não linear para produzir a saída final



Adaptado de [LeCun *et al.*, 2015]

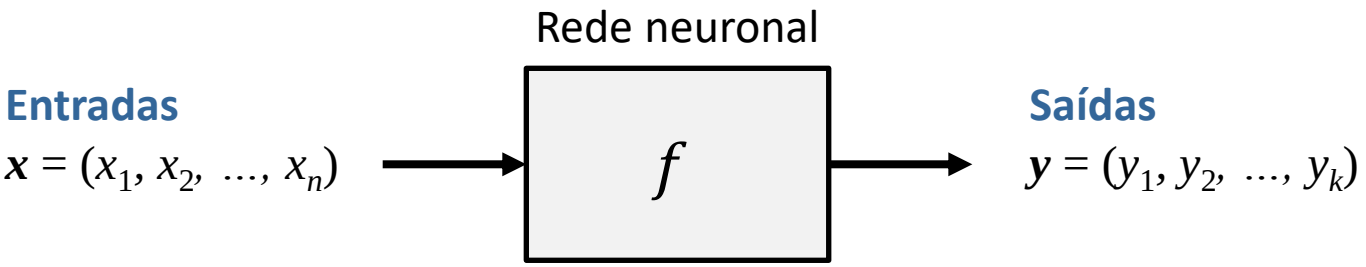
Neste exemplo, existem dois neurónios de entrada, dois neurónios escondidos e um neurónio de saída, mas as redes utilizadas em aplicações reais podem conter milhões de neurónios, como é o caso dos grandes modelos de linguagem mais recentes, o que possibilita a aplicação de redes neuronais multicamada para resolução de problemas de grande complexidade

REDES NEURONAIS ARTIFICIAIS

Representação formal

- Uma rede neuronal artificial pode ser representada de modo formal for uma função de transferência f na forma:

$$f\colon \mathbb{R}^n \rightarrow \mathbb{R}^k$$



Capacidade computacional

- A capacidade computacional de uma rede neuronal artificial para resolver problemas de natureza geral pode ser definida em termos do tipo de funções que a rede tem capacidade de aproximar

TEOREMA DA APROXIMAÇÃO UNIVERSAL

Enunciado do teorema

- Seja $f(x)$ a função de transferência de uma rede neuronal, a qual representa a saída da rede $y = f(x)$ para a entrada x
- Seja $g(x)$ uma função contínua definida num intervalo compacto

  $[a, b] \subset \mathbb{R}$
- Então, para qualquer valor $\varepsilon > 0$ existe uma rede neuronal com uma única camada escondida, contendo um número finito de neurónios, que pode aproximar $g(x)$ com precisão ε, ou seja:

  $| g(x) - f(x) | < \varepsilon, \forall x \in [a, b]$

REDES NEURONAIS MULTICAMADA

Capacidade Computacional

- **Teorema da Aproximação Universal**
  - Uma rede neuronal com uma única camada escondida, contendo um número finito de neurónios, pode aproximar qualquer função contínua num intervalo compacto (limitado e fechado) em $\mathbb{R}^n$, com precisão arbitrária, desde que a função de activação dos neurónios escondidos seja não linear
- Ou seja, uma rede neuronal com uma única camada escondida é um aproximador universal de funções
  - Dado um número de neurónios suficiente e valores adequados dos pesos das ligações é possível aproximar qualquer função
  - **Problema:**
    - Qual a arquitectura da rede e os pesos adequados?

REDES NEURONAIS MULTICAMADA

Implicações do Teorema da Aproximação Universal

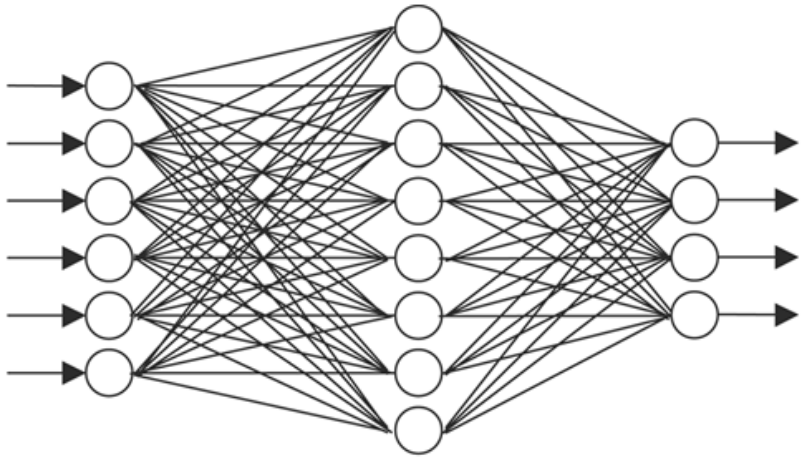
- **Capacidade computacional**
  - As redes neuronais com uma única camada escondida são aproximadoras universais de funções
- **Não Linearidade**
  - A condição de a função de activação dos neurónios escondidos ser não linear é essencial para a capacidade de aproximação universal
- **Arquitectura da rede**
  - O teorema não especifica o número concreto de neurónios necessários na camada escondida, apenas indica que um número finito de neurónios é suficiente
  - Na prática, o número de neurónios necessários depende da complexidade da função a ser aproximada
- **Problema da aprendizagem**
  - O teorema garante a existência de uma rede neuronal que pode aproximar a função adequada à resolução de um problema, mas não fornece um método de aprendizagem específico

REDES NEURONAIS MULTICAMADA

Caso geral

Redes com pelo menos uma camada escondida são aproximadores universais de funções

Nas camadas escondidas da rede são formadas representações distribuídas das principais características dos dados, o que possibilita a **abstracção** e **generalização** dessas características

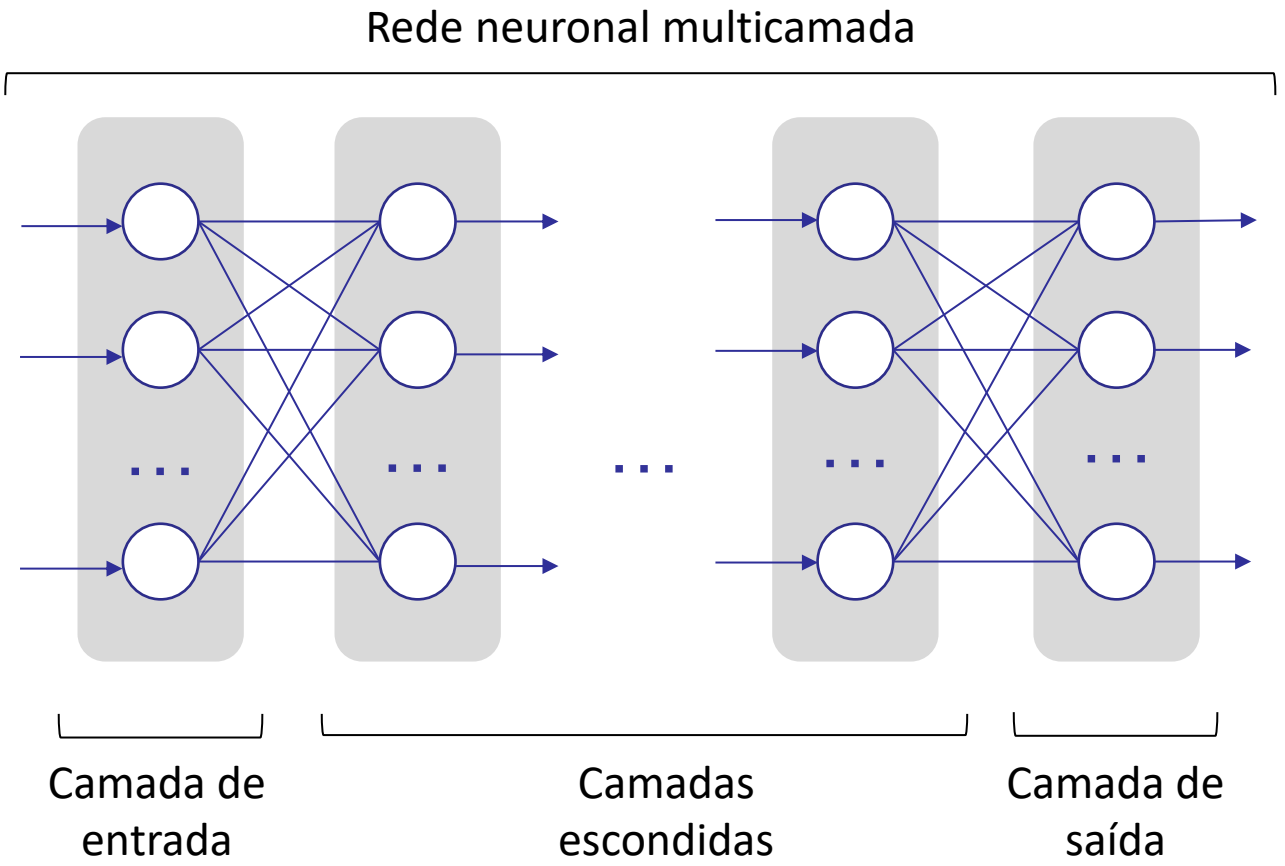


No entanto, pelo facto de uma rede neuronal multicamada teoricamente ter capacidade computacional para aproximar qualquer tipo de função, isso não significa que se tenha o conhecimento concreto de como fazer

- **Problema da concepção e treino de redes neuronais artificiais**

REDES NEURONAIS MULTICAMADA

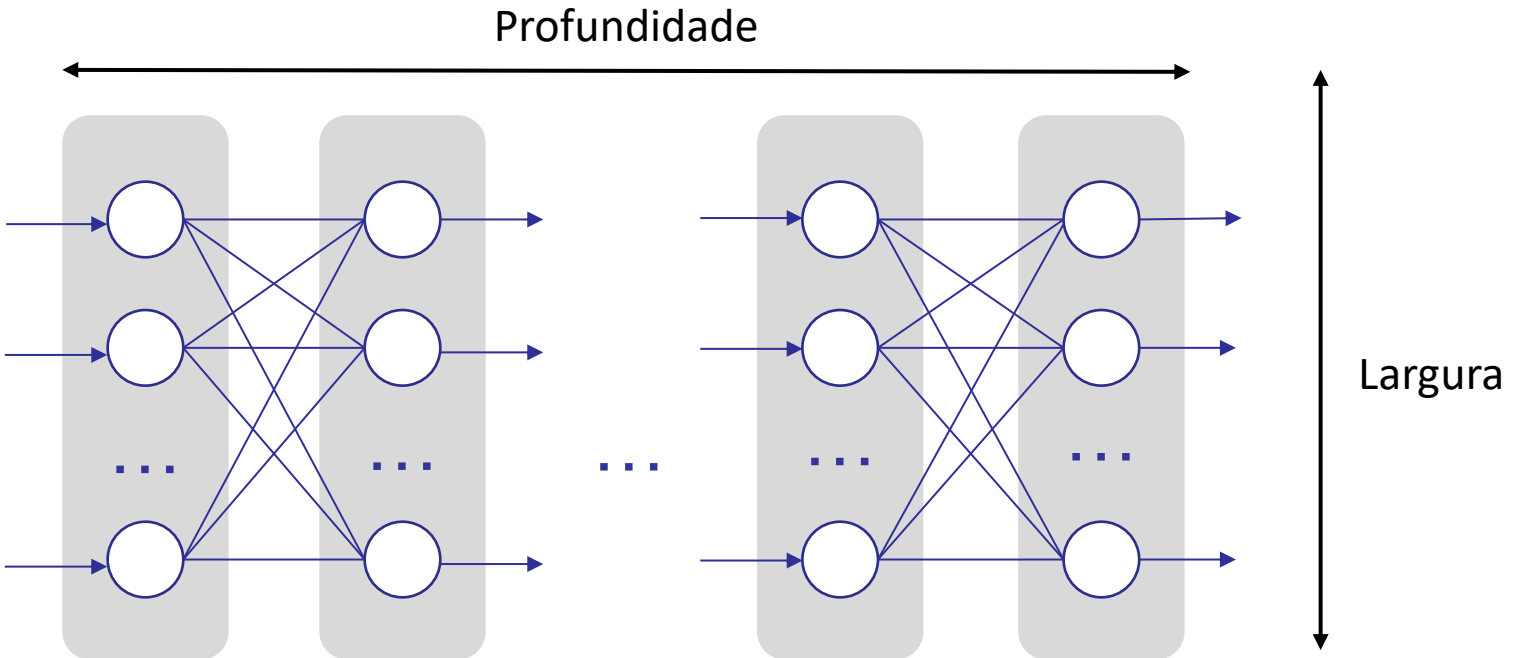
No caso geral, uma rede neuronal artificial pode ter múltiplas camadas com características diferenciadas, nomeadamente, em termos de tipos de neurónios e de conectividade entre neurónios, sendo um exemplo as *redes neuronais profundas*



REDES NEURONAIS MULTICAMADA

Características de uma rede neuronal multicamada

- **Profundidade**
  - Número de camadas da rede
- **Largura** (dimensão da camada)
  - Número de neurónios por cada camada



CAMADAS DE UMA REDE NEURONAL

Processamento de cada camada

- **Recepção das entradas x**
  - Os neurónios da camada recebem as saídas dos neurónios da camada anterior como entradas
- **Cálculo da soma de activação h**
  - Cada neurónio calcula o produto interno entre as suas entradas e os respectivos pesos sinápticos
- **Aplicação da função de activação φ**
  - A soma de activação é aplicada à função de activação do neurónio, produzindo uma resposta não linear
- **Disponibilização das saídas y**
  - As saídas resultantes da função de activação são enviadas como entradas para a próxima camada da rede

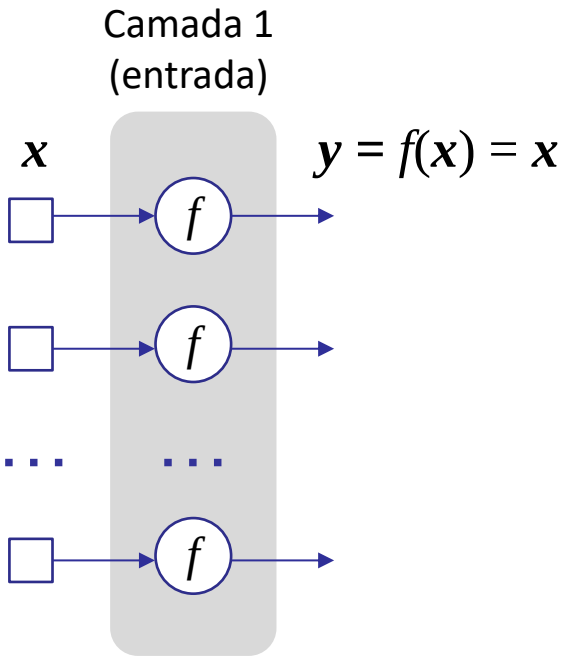
CAMADAS DE UMA REDE NEURONAL

Camada de entrada

A camada de entrada regista e disponibiliza os dados de entrada

É representada por:

- Um vector de saída $\boldsymbol{y}$ equivalente ao vector de entrada $\boldsymbol{x}$
- Os respectivos neurónios implementam a função identidade $f(x_i) = x_i$

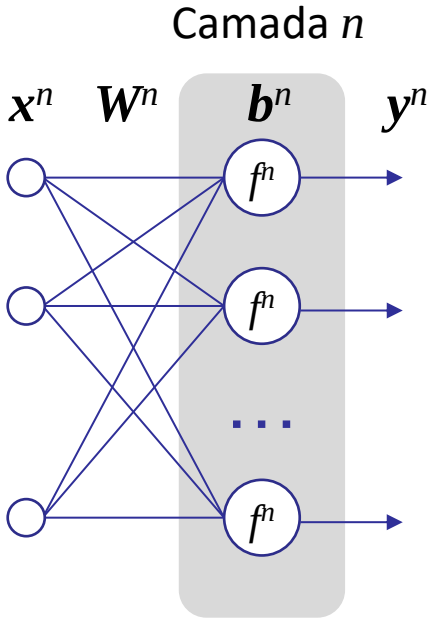


CAMADAS DE UMA REDE NEURONAL

Camadas escondidas e de saída

Cada camada n escondida ou de saída é representada por:

- Uma matriz $\boldsymbol{W}^n$ de pesos das ligações dos respectivos neurónios com os neurónios da camada anterior
- Uma matriz $\boldsymbol{b}^n$ de pendores dos neurónios
- Uma função de activação $\boldsymbol{\varphi}^n$
- O vector de saída $\boldsymbol{y}^n$ é gerado por aplicação da função de activação à soma ponderada das entradas pelos respectivos pesos e somados os pendores (produto escalar entre o vector de entrada $\boldsymbol{x}^n = \boldsymbol{y}^{n-1}$ e a matriz $\boldsymbol{W}^n$ e adicionado o vector de pesos $\boldsymbol{b}^n$), à qual é aplicada a função de activação φ^n



$$\boldsymbol{h}^n = \boldsymbol{x}^n \boldsymbol{W}^n + \boldsymbol{b}^n$$

$$\boldsymbol{y}^n = f(\boldsymbol{x}^n) = \varphi^n(\boldsymbol{h}^n)$$

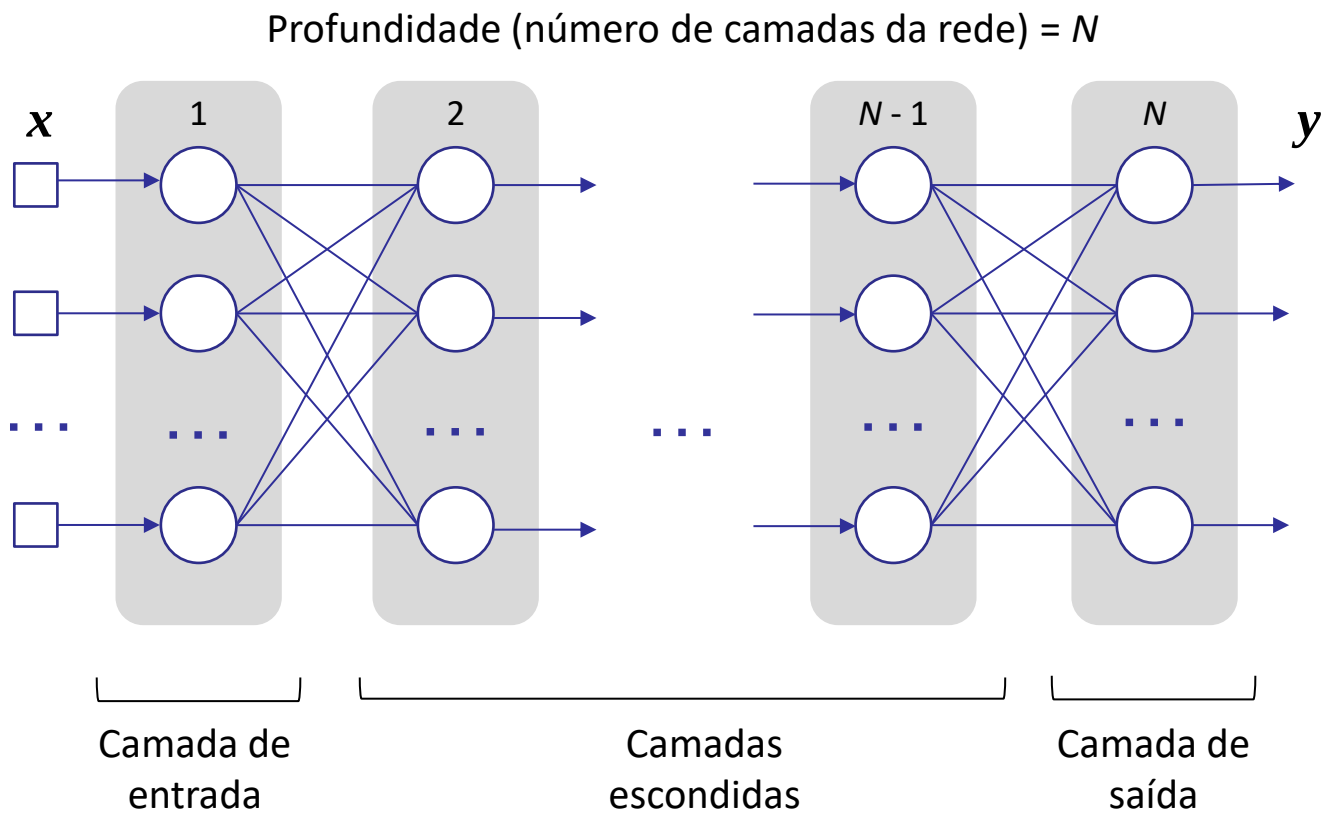
$$h_k{}^n = \underbrace{\sum_{j=1}^{m} x_j{}^n w_{jk}{}^n}_{\text{Multiplicação de um vector por uma matriz}} + b_k{}^n$$

REDES NEURONAIS MULTICAMADA

Representação no formato matricial

Os pesos e pendores são designados ***parâmetros*** da rede, sendo representados pelo conjunto Θ de pesos $\boldsymbol{W}^n$ e pendores $\boldsymbol{b}^n$ com $n > 1$ (não inclui a camada de entrada)

Parâmetros da rede: $\theta = \{\boldsymbol{W}^2, \boldsymbol{b}^2, \ldots, \boldsymbol{W}^{N-1}, \boldsymbol{b}^{N-1}, \boldsymbol{W}^N, \boldsymbol{b}^N\}$

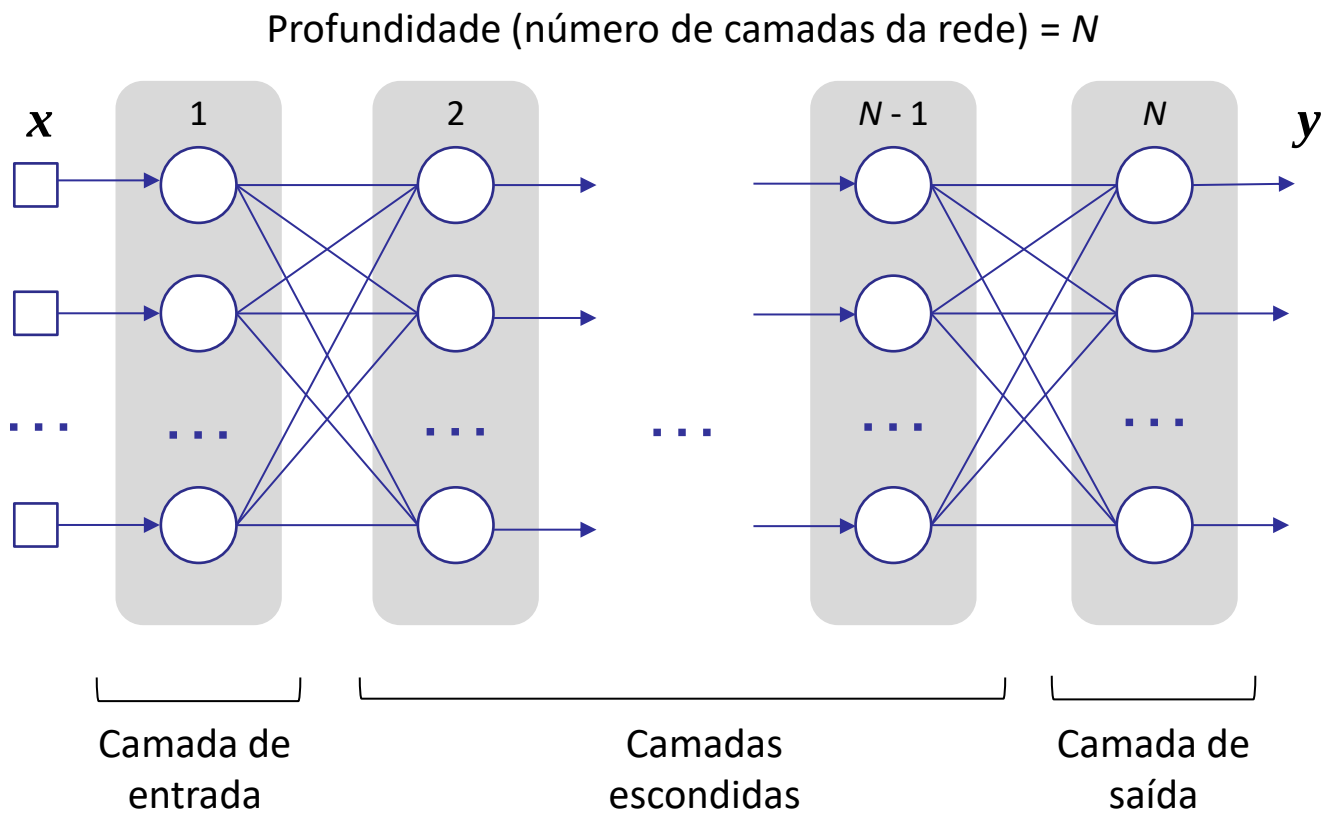


REDES NEURONAIS MULTICAMADA

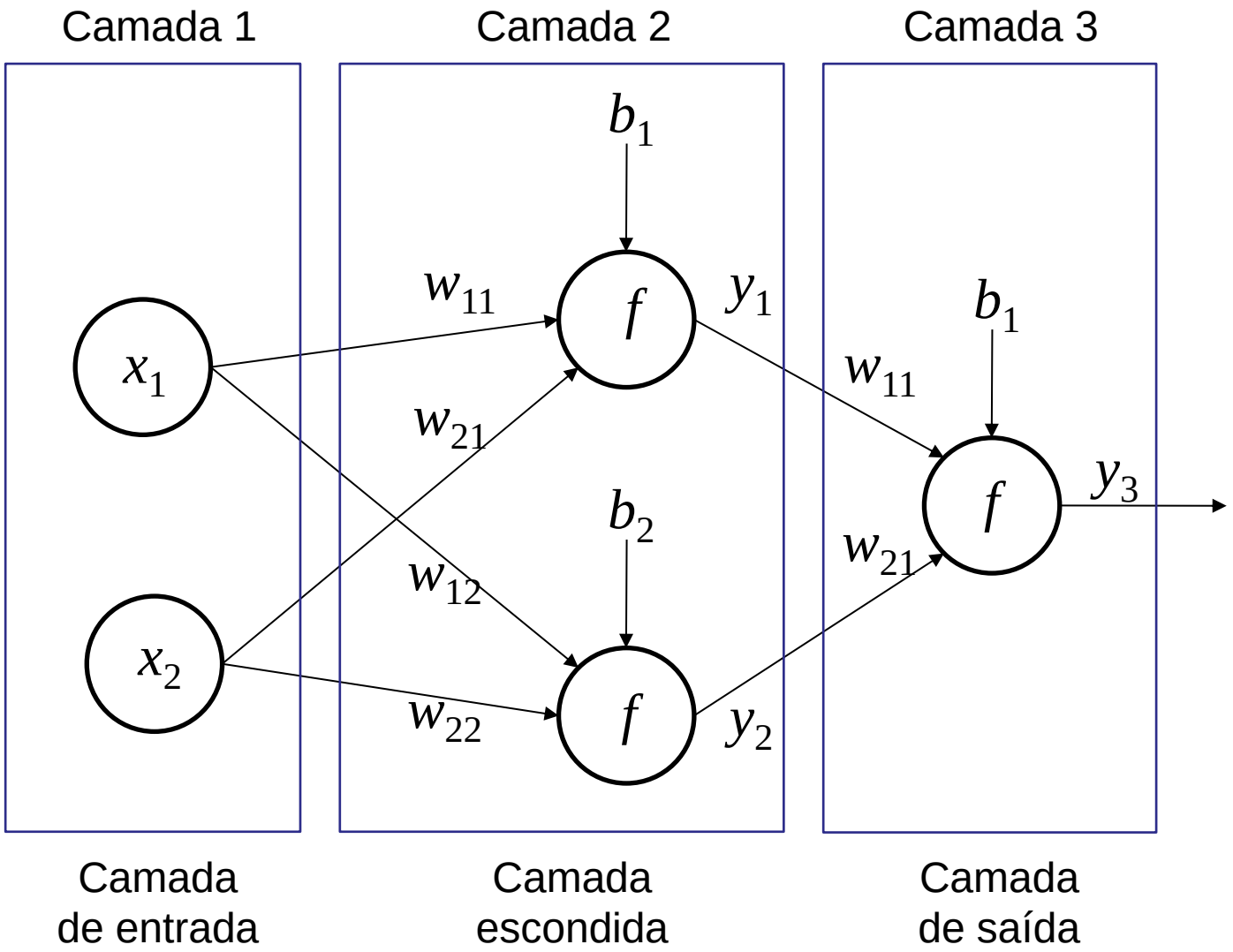
Activação directa da rede

A activação directa da rede produz a saída $\boldsymbol{y}$ da rede com base na entrada $\boldsymbol{x}$ por activação sequencial encadeada das várias camadas da rede

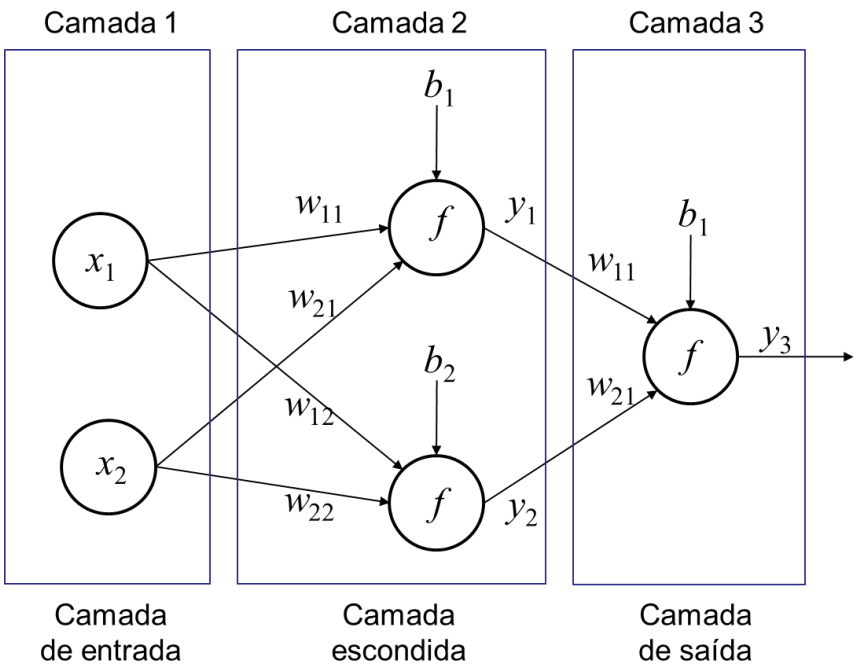
$$\boldsymbol{y}^n = f^n(\boldsymbol{x}^n) = \varphi^n(\boldsymbol{h}^n);\ \ \boldsymbol{h}^n = \boldsymbol{x}^n\boldsymbol{W}^n + \boldsymbol{b}^n;\ \ \text{com } \boldsymbol{x}^n = \boldsymbol{y}^{n-1} \text{ para } n > 1 \text{ e } \boldsymbol{y}^1 = \boldsymbol{x}$$



EXEMPLO: REDE XOR



EXEMPLO: REDE XOR



Parâmetros da rede

Camada 2

$\boldsymbol{W}^2$

1	-1
-1	1

$\boldsymbol{b}^2$

-0,5	-0,5

Camada 3

$\boldsymbol{W}^3$

1
1

$\boldsymbol{b}^3$

-0,5

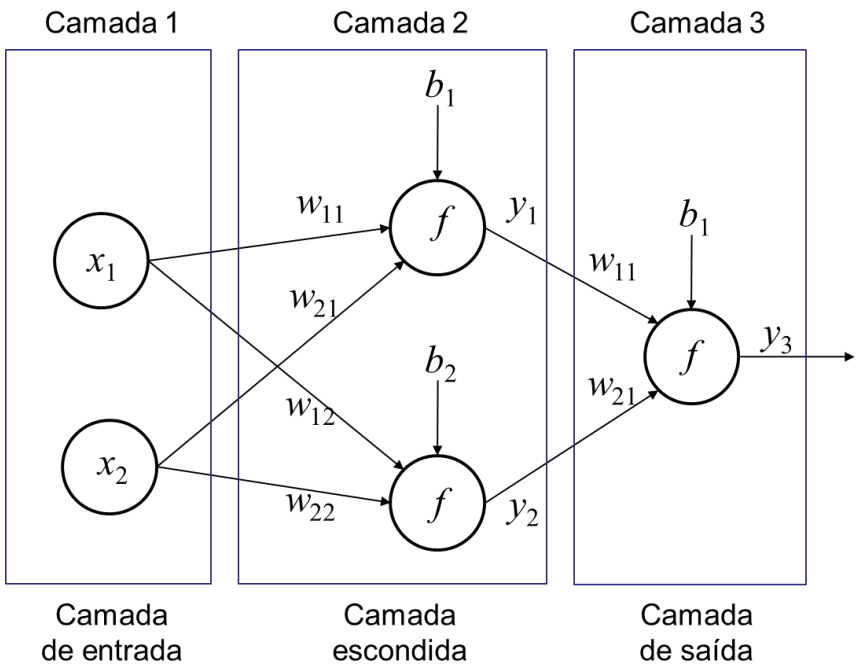
Função de activação

$$f(x) = \text{degrau}(x) = \begin{cases} 1 \text{ se } x > 0 \\ 0 \text{ se } x \leq 0 \end{cases}$$

EXEMPLO: REDE XOR

Função XOR

A	B	A⊕B
0	0	0
0	**1**	**1**
1	0	1
1	1	0



$$f(x) = \begin{cases} 1 \text{ se } x > 0 \\ 0 \text{ se } x \leq 0 \end{cases}$$

$$\boldsymbol{h}^n = \boldsymbol{x}^n \boldsymbol{W}^n + \boldsymbol{b}^n$$

$$h_k{}^n = \sum_{j=1}^{m} x_j{}^n w_{jk}{}^n + b^n$$

$$\boldsymbol{y}^n = f(\boldsymbol{x}^n) = \varphi^n(\boldsymbol{h}^n)$$

Activação directa da rede

Camada 1 → Camada 2

$\boldsymbol{x} = (\mathbf{0}, \mathbf{1})$

$\boldsymbol{y}^1 = \boldsymbol{x} = (0, 1)$

$\boldsymbol{W}^2$

1	-1
-1	1

$\boldsymbol{b}^2$

-0,5	-0,5

$\boldsymbol{x}^2 = \boldsymbol{y}^1 = (0, 1)$

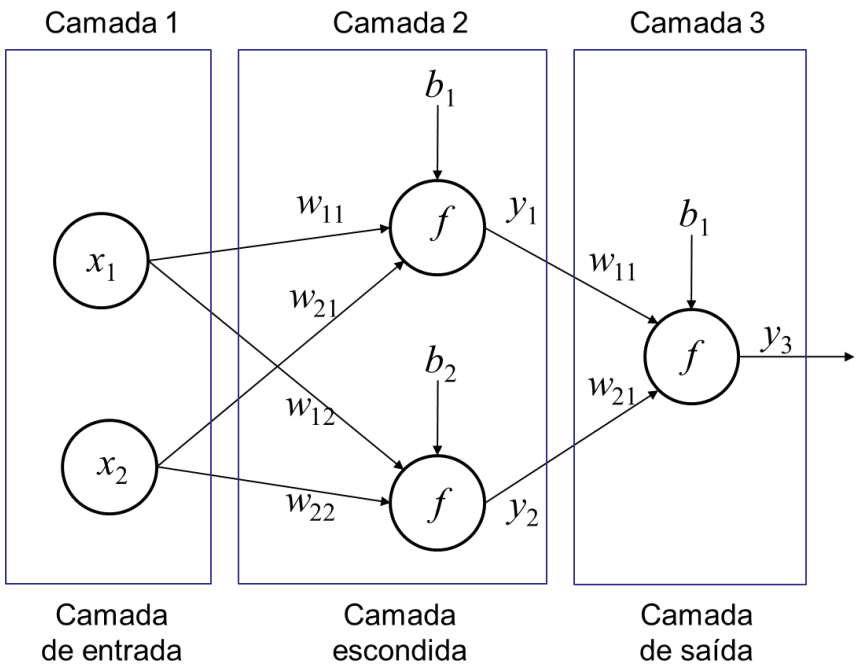
$\boldsymbol{h}^2 = (0*1 + 1*-1 + -0.5,\ 0*-1 + 1*1 + -0.5) = (-1.5, 0.5)$

$\boldsymbol{y}^2 = (0, 1)$

EXEMPLO: REDE XOR

Função XOR

A	B	A⊕B
0	0	0
0	**1**	**1**
1	0	1
1	1	0



$$f(x) = \begin{cases} 1 \text{ se } x > 0 \\ 0 \text{ se } x \leq 0 \end{cases}$$

$$\boldsymbol{h}^n = \boldsymbol{x}^n \boldsymbol{W}^n + \boldsymbol{b}^n$$

$$h_k{}^n = \sum_{j=1}^{m} x_j{}^n w_{jk}{}^n + b^n$$

$$\boldsymbol{y}^n = f(\boldsymbol{x}^n) = \varphi^n(\boldsymbol{h}^n)$$

Activação directa da rede

Camada 2 → Camada 3

$\boldsymbol{y}^2 = (0, 1)$

$\boldsymbol{W}^3$	$\boldsymbol{b}^3$
1	-0,5
1	

$\boldsymbol{x}^3 = \boldsymbol{y}^2 = (0, 1)$

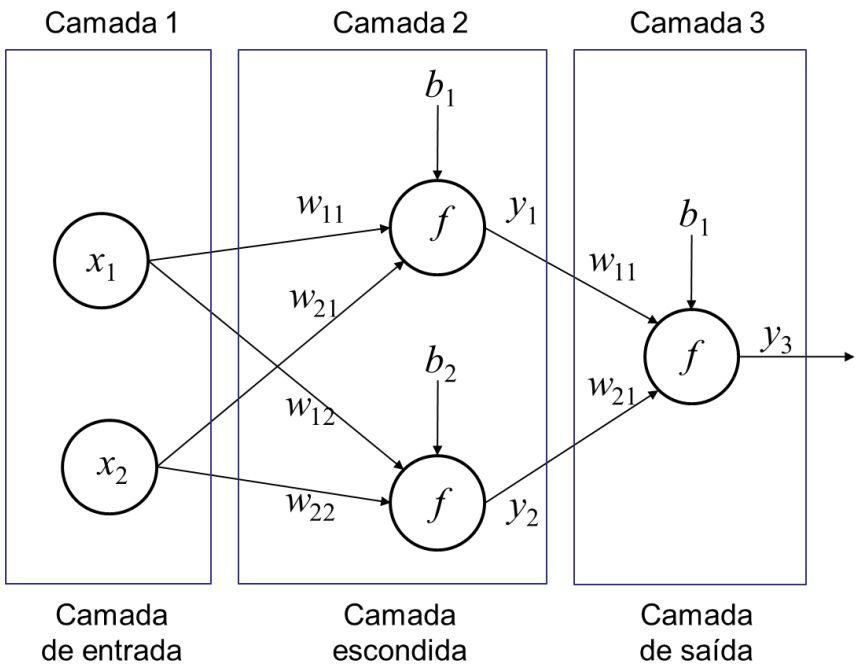
$\boldsymbol{h}^3 = (0*1 + 1*1 + -0.5) = (0.5)$

$\boldsymbol{y}^3 = (\mathbf{1})$

EXEMPLO: REDE XOR

Função XOR

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	**1**	**0**



$$f(x) = \begin{cases} 1 \text{ se } x > 0 \\ 0 \text{ se } x \leq 0 \end{cases}$$

$$\boldsymbol{h}^n = \boldsymbol{x}^n \boldsymbol{W}^n + \boldsymbol{b}^n$$

$$h_k{}^n = \sum_{j=1}^{m} x_j{}^n w_{jk}{}^n + b^n$$

$$\boldsymbol{y}^n = f(\boldsymbol{x}^n) = \varphi^n(\boldsymbol{h}^n)$$

Activação directa da rede

Camada 1 → Camada 2

$\boldsymbol{x} = (1, 1)$

$\boldsymbol{y}^1 = \boldsymbol{x} = (1, 1)$

$\boldsymbol{W}^2$

1	-1
-1	1

$\boldsymbol{b}^2$

-0,5	-0,5

$\boldsymbol{x}^2 = \boldsymbol{y}^1 = (1, 1)$

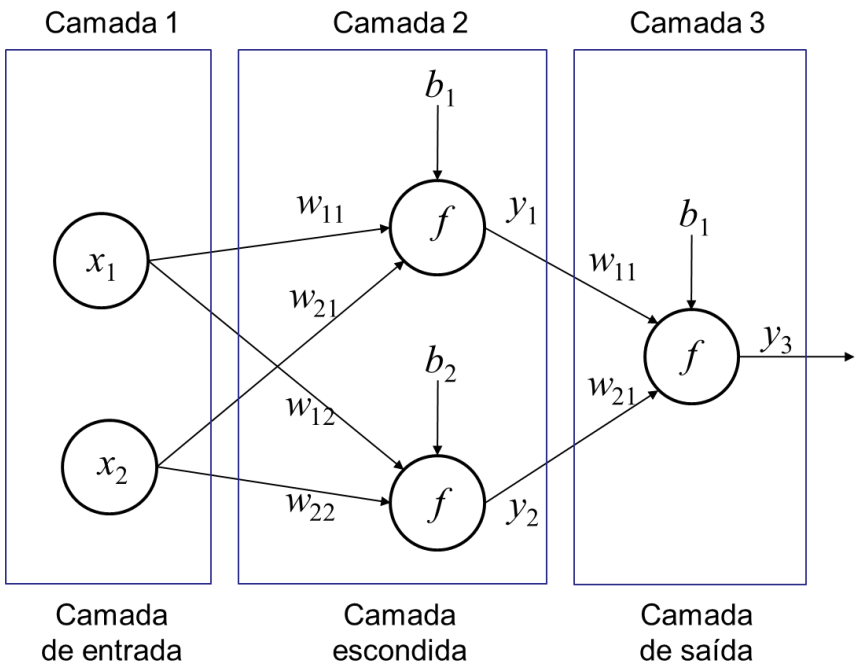
$\boldsymbol{h}^2 = (1*1 + 1*-1 + -0.5, 1*-1 + 1*1 + -0.5) = (-0.5, -0.5)$

$\boldsymbol{y}^2 = (0, 0)$

EXEMPLO: REDE XOR

Função XOR

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	**1**	**0**



$$f(x) = \begin{cases} 1 \text{ se } x > 0 \\ 0 \text{ se } x \leq 0 \end{cases}$$

$$\boldsymbol{h}^n = \boldsymbol{x}^n \boldsymbol{W}^n + \boldsymbol{b}^n$$

$$h_k{}^n = \sum_{j=1}^{m} x_j{}^n w_{jk}{}^n + b^n$$

$$\boldsymbol{y}^n = f(\boldsymbol{x}^n) = \varphi^n(\boldsymbol{h}^n)$$

Activação directa da rede

Camada 2 → Camada 3

$\boldsymbol{y}^2 = (0, 0)$

$\boldsymbol{W}^3$	$\boldsymbol{b}^3$
1	-0,5
1	

$\boldsymbol{x}^3 = \boldsymbol{y}^2 = (0, 0)$

$\boldsymbol{h}^3 = (0*1 + 0*1 + -0.5) = (-0.5)$

$\boldsymbol{y}^3 = (\mathbf{0})$

REDES NEURONAIS MULTICAMADA

Características principais

- **Representação de relações não lineares complexas**
  - Devido a múltiplas camadas escondidas e a funções de activação não lineares, as redes neuronais multicamada podem representar relações não lineares arbitrariamente complexas entre as entradas e as saídas
- **Extração de características**
  - As camadas escondidas podem representar características sucessivamente mais abstractas dos dados de entrada, possibilitando um adequado desempenho mesmo perante problemas complexos
- **Tolerância a ruído**
  - As redes neuronais multicamada são relativamente robustas a dados com ruído ou incompletos, devido à sua capacidade de abstracção e generalização e ao facto das representações dos dados serem distribuídas
- **Processamento distribuído**
  - O processamento em cada camada pode ser realizado de forma distribuída, possibilitando grande eficiência computacional

REDES NEURONAIS MULTICAMADA

Características principais

- **Flexibilidade**
  - As redes neuronais multicamada podem ser aplicadas para resolução de diferentes tipos de problemas, desde problemas de aprendizagem supervisionada, como problemas de classificação, até problemas de aprendizagem não-supervisionada
- **Escalabilidade**
  - As redes neuronais multicamada podem ser dimensionadas para resolver problemas de grande complexidade, através do aumento do número de camadas e de neurónios
- **Eficiência Computacional**
  - O paralelismo inerente às redes neuronais multicamada torna-as computacionalmente eficientes, especialmente em plataformas de hardware com capacidade de processamento distribuído

REDES NEURONAIS MULTICAMADA

Problemas principais

- **Dificuldade de interpretação das configurações das redes**
  - As redes neuronais representam conhecimento através de configurações numéricas massivamente distribuídas e de grande complexidade, tornando extremamente difícil compreender o significado dessas representações e de como são produzidos os respectivos resultados
- **Definição da arquitectura de rede**
  - Métodos sistemáticos de definição da melhor arquitectura de rede neuronal para um determinado problema são muito limitados, requerendo tipicamente experiência e ajuste experimental de hiper-parâmetros da rede
- **Necessidade de grandes conjuntos de dados**
  - As redes neuronais multicamada requerem grandes quantidades de dados para o respectivo treino, os quais podem ser difíceis de obter e ter elevado custo
  - No caso da aprendizagem supervisionada, por exemplo, para problemas de classificação, acresce a necessidade de etiquetagem dos dados

REDES NEURONAIS MULTICAMADA

Problemas principais

- **Tempo de treino**
  - O treino de redes neuronais complexas, nomeadamente de grande profundidade, pode levar muito tempo, em particular com grandes conjuntos de dados
- **Sensibilidade a ruído**
  - Apesar da relativa robustez em relação a dados com ruído ou incompletos, as redes neuronais podem ser vulneráveis a exemplos enganadores, que incluem ruído específico para perturbar o seu adequado funcionamento

NEURÓNIO - IMPLEMENTAÇÃO

Informação associada a um neurónio e respectivos valores iniciais

1.	$\text{iniciar}(d, \varphi)$:	
2.	$d \in Z^+$	Dimensão de entrada (número ligações de entrada)
3.	$\varphi: R \rightarrow R$	Função de activação
4.	$\boldsymbol{w} = \text{random}(d, -1, 1)$	Vector de pesos das ligações de entrada
5.	$b = \text{random}(-1, 1)$	Pendor interno do neurónio
6.	$h = 0$	Soma da activação das entradas
7.	$y = 0$	Saída do neurónio

NEURÓNIO

Propagar entradas gerando a saída do neurónio

1. propagar($\boldsymbol{x}$):
2. $h = \boldsymbol{w} \cdot \boldsymbol{x} + b$
3. $y = \varphi(h)$
4. return y

$\boldsymbol{x}$ - vector de entradas

Produto escalar

$$\boldsymbol{a} \cdot \boldsymbol{b} = \sum_{i=1}^{n} a_i b_i = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n$$

CAMADA DE ENTRADA

Informação associada a uma camada de entrada e respectivos valores iniciais

1. $\text{iniciar}(d_s)$:
2. $d_s \in Z^+$ — Dimensão de saída (número de saídas da camada)
3. $\boldsymbol{y} = [0 \text{ for } d_s]$

Propagar entrada gerando a saída da camada

1. $\text{propagar}(\boldsymbol{x})$:
2. $\boldsymbol{y} = \boldsymbol{x}$ — Manter informação de saída
3. return $\boldsymbol{y}$

$\boldsymbol{x}$ Vector de entrada

$\boldsymbol{y}$ Vector de saída

CAMADA DENSA

Informação associada a uma camada densa e respectivos valores iniciais

1. $\text{iniciar}(d_e, d_s, \varphi)$:
2. $d_e \in Z^+$ — Dimensão de entrada (número ligações de entrada)
3. $d_s \in Z^+$ — Dimensão de saída (número de saídas da camada)
4. $\varphi: R \rightarrow R$ — Função de activação
5. $neuronios = [\text{Neuronio}(d_e, \varphi) \text{ for } d_s]$ — Neurónios da camada

Saída da camada

1. $\boldsymbol{y}$:
2. return $[neuronio.y \text{ for } neuronio \text{ in } neuronios]$

CAMADA DENSA

Propagar entrada gerando a saída da camada

1. propagar($\boldsymbol{x}$):
2. $\boldsymbol{y} = [neuronio.\text{propagar}(\boldsymbol{x}) \text{ for } neuronio \text{ in } neuronios]$
3. return $\boldsymbol{y}$

$\boldsymbol{x}$ Vector de entrada

$\boldsymbol{y}$ Vector de saída

REDE NEURONAL

Informação associada a uma rede neuronal e respectivos valores iniciais

1. iniciar($forma, \varphi$):
2. $camadas = [\,]$
3. $N = \text{length}(forma)$
4. $d_s^1 = forma[1]$ — Dimensão de entrada
5. $camada^1 = \text{CamadaEntrada}(d_s^1)$ — Camada de entrada
6. $camadas.\text{append}(camada^1)$
7. for $n = 2$ to N:
8. $\quad d_e^n = forma[n-1]$
9. $\quad d_s^n = forma[n]$
10. $\quad camada^n = \text{CamadaDensa}(d_e^n, d_s^n, \varphi)$
11. $\quad camadas.\text{append}(camada^n)$

$forma$	Forma da rede (lista de número de neurónios por camada)
φ	Função de activação dos neurónios das camadas
N	Número de camadas
d_e^n	Dimensão de entrada da camada n
d_s^n	Dimensão de saída da camada n

REDE NEURONAL

Prever saídas para um conjunto de entradas

```
1.  prever(X):
2.      Y = [propagar(x) for x ∈ X]
3.      return Y
```

$\boldsymbol{X}$ Conjunto de vectores de entrada

$\boldsymbol{Y}$ Conjunto de vectores de saída

$\boldsymbol{x}$ Vector de entrada

Propagar saída para uma dada entrada

```
1.  propagar(x):
2.      y = x
3.      for camada in camadas:
4.          y = camada.propagar(y)
5.      return y
```

BIBLIOGRAFIA

[Aggarwal, 2018]
C. Aggarwal, Neural Networks and Deep Learning
Springer, 2018

[Munakata, 1998]
T. Munakata, *Fundamentals of the New Artificial Intelligence*, Springer, 1998

[Winston, 1992]
P. Winston, *Artificial Intelligence*, 3rd Edition, Addison-Wesley, 1992

[Raizer *et al.*, 2009]
K. Raizer, H. Idagawa, E. Nobrega, L. Ferreira, *Training and Applying a Feedforward Multilayer Neural Network in GPU*, CILAMCE, 2009

[Gutierrez-Osuna, 2005]
R. Gutierrez-Osuna*, Introduction to Pattern Analysis*, Texas A&M University, 2005

[LeCun *et al.*, 2015]
Y. LeCun, Y. Bengio, G. Hinton, *Deep learning*, Nature, Vol. 521, p. 436-444, 2015

[Deisenroth *et al*., 2020]
M. Deisenroth, A. Faisal, Cheng Soon Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020